

Assunto: Listas, Tuplas e o ciclo for.

Número da Aula: 04

Elaborado por: Heli Romeu Miguel Sanguene

Sumário

1. Introdução	1
2. Listas	2
2.1. Principais operações sobre as listas.....	2
2.1.1. Adicionar elemento(s) a uma lista.....	2
2.1.2. Remover elementos de uma lista	3
2.1.3. Slicing.....	4
2.1.4. Compreensão de lista	6
2.1.5. Atribuição.....	7
2.2. Outras operações sobre as listas.....	7
2.2.1. Concatenação e repetição.....	7
2.2.2. Aninhamento (nesting)	7
2.2.2. Funções de uso comum sobre as listas	8
3. Ciclo for e a função range()	9
3.1. Ciclo for.....	9
3.2. função range()	10
4. Tuplas	10
4.1. Conversão de lista para tupla e vice-versa	11
Bibliografia.....	11

1. Introdução

O dia-a-dia de um programador é cheio de desafios, tais desafios costumam ser para resolver problemas do quotidiano com o intuito de enquadrá-los em soluções representáveis por uma máquina, mais precisamente um computador. Uma das situações frequentes acontece quando se pretende fazer agrupamento de valores ou objectos frequentemente utilizados no “mundo real”. Para resolver isso, o python possui uma variedade de estruturas que permitem coleccionar valores e também facilitar as operações sobre esses valores de forma individual e colectiva.

Desta feita, nesta aula serão abordadas duas soluções para coleccionar valores de forma sequencial: as listas e tuplas; bem como as operações básicas para tirar o maior proveito das mesmas estruturas durante o desenvolvimento de softwares na linguagem python.

A aula, destina-se também em abordar com alguma brevidade o ciclo *for*, que em python é praticamente desenhado para percorrer colecções.

2. Listas

É uma coleção de itens numa ordem particular, onde o tipo de cada item é irrelevante, permitindo que cada elemento seja alterado ao longo da execução do programa.

Uma lista pode ser criada através da seguinte sintaxe:

```
<nomeDaVariavel> = [<element1>, <elemento2>, ..., <elemento>]
```

Para se criar uma lista, o lado direito da atribuição seria suficiente. Mas, geralmente essa criação é acompanhada de uma atribuição (não obrigatória). Uma lista pode ser criada sem nenhum elemento, ou seja, uma lista vazia.

```
nomes = ['João ', 'André', 'José', 'Tomás', 100]
print(nomes[0])
print(nomes[-1])
```

Uma lista permite o acesso individual a cada um dos elementos da mesma pelo operador de indexação []. Um facto interessante, está no facto de o python permitir esse acesso para índices com valores negativos.

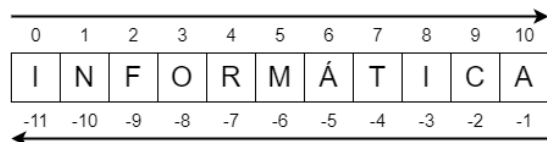


Figura 1. Diferentes formas de acesso aos elementos de uma sequência.

Para retornar o tamanho de uma lista utiliza-se a função `len()`.

```
nomes = ['João', 'André', 'José', 'Tomás']
len(nomes)
```

2.1. Principais operações sobre as listas

Existe um conjunto de operações que podem ser executadas sobre as listas, que as tornam úteis para situações práticas do quotidiano de um programador. Elas estão principalmente agrupadas em adição, remoção.

2.1.1. Adicionar elemento(s) a uma lista

O python permite três principais métodos: `.append()`, `.insert()` e `.extend()`.

O método `.append()` permite adicionar elementos ao final da lista, alterando assim o tamanho da mesma. A sua sintaxe é:

```
<nomeDaLista>.append(<novoElemento>)
```

```
nomes = ['João']
nomes.append('José')
nomes.append('Filipe')
nomes.append('Tomás')
```

Na lista anterior, inicialmente havia apenas um elemento, os demais elementos passaram a ser anexados sequencialmente à mesma pelo método `append()`. A lista, teria então os seguintes elementos: ['João', 'José', 'Filipe', 'Tomás'].

O método `.insert()` permite adicionar um novo elemento numa posição específica da lista. A sua sintaxe é:

```
<nomeDaLista>.insert(<posição>, <novoElemento>)
```

Porém existem dois cenários a considerar:

- Se a <posição> indicada já estiver ocupada por determinado valor, acontecerá um deslocamento de uma posição para toda a lista, fazendo assim a adição na posição desejada. O elemento actual não será removido; ou seja, se ele estiver na posição i , passará para posição $i+1$ e a posição i passará a ser ocupada pelo novo elemento.
- Se a posição indicada estiver fora do intervalo de índices da lista (por exemplo, posição 10 numa lista com apenas cinco elementos), o novo elemento será adicionado ao final da lista.

```
nomes = ['João', 'André', 'José', 'Tomás']  
nomes.insert(0, 'Daniel')
```

A lista, teria então a seguinte estrutura: ['Daniel', 'João', 'André', 'José', 'Tomás']. Dessa forma, o valor 'João' passa a ocupar a posição 1, ao invés de 0 e o valor 'André' a posição 2 ao invés de 1 e assim sucessivamente.

O método `.extend()` permite adicionar um “iterável” no final da lista. Um “iterável” é toda estrutura que pode ser percorrida naturalmente por um ciclo (maioritariamente o ciclo *for*). Como uma lista é uma iterável, então pode-se estender uma lista através de outra. A sua sintaxe é:

```
<nomeDaLista>.extend(<iterável>)
```

```
nomes = ['João', 'André', 'José', 'Tomás']  
nomes.extend(['Josefa', 'Andrea'])
```

A lista utilizada para o método `.extend()`, podia ser armazenada numa variável e posteriormente essa variável servir de base para estender a lista. Dessa forma a lista final seria: ['João', 'André', 'José', 'Tomás', 'Josefa', 'Andrea'].

2.1.2. Remover elementos de uma lista

O python fornece pelo menos três métodos para remover elementos de uma lista: `.pop()`, `.remove()` e o operador **del**.

del

O operador **del** é usado antes da lista, e deve incluir a posição do elemento a ser eliminado.

```
del <nomeDaLista>[<posição>]
```

```
nomes = ['João', 'André', 'José', 'Tomás']  
del nomes[1]
```

Dessa maneira, seria eliminado o elemento na posição 1, cujo valor era 'André'. Os demais elementos se auto-ajustariam tomando novas posições cobrindo a posição do elemento que já não existe. Caso <posição> esteja fora dos limites de índice da lista, o python emitirá um erro.

.pop()

Existem situações onde é bastante útil retornar o elemento eliminado. Para tais situações, utiliza-se o método `.pop()`, com a seguinte sintaxe:

```
<nomeDaLista>.pop(<posição>)
```

```
nomes = ['João', 'André', 'José', 'Tomás']  
eliminado = nomes.pop()
```

Porém, o `.pop()` apenas elimina o último elemento da lista. Dessa maneira, o trecho anterior eliminará apenas o último da lista e será atribuído à variável *eliminado*.

Quando se desejar eliminar um elemento em qualquer posição, basta incluir a posição do elemento e o mesmo será removido da lista.

```
nomes = ['João', 'André', 'José', 'Tomás']  
eliminado = nomes.pop(1)
```

.remove()

O método `remove()` permite eliminar um elemento pelo valor especificado. Tecnicamente acontecerá uma busca preliminar, se o elemento for encontrado será eliminado. Esse método apenas eliminará a primeira ocorrência do valor.

`<nomeDaLista>.remove(<valor>)`

```
nomes = ['João', 'André', 'José', 'Tomás']  
nomes.remove('André')
```

Dessa maneira, o python vai verificar se o elemento se encontra na lista. Se for encontrado, será eliminado. Caso contrário, o python emitirá um erro de execução.

2.1.3. Slicing

O *slicing* basicamente consiste num conjunto de operações que permitem ter acesso a determinadas partes (ou intervalos) de uma lista por meio da indexação, mais precisamente com o operador `[]`.

Essas operações são executadas às listas através do uso de *dois pontos* (`:`). Algumas operações frequentes são:

`[:]` - permite criar uma sublista com todos os elementos da lista de base.

`[:<fim>]` - permite criar uma sublista do primeiro elemento até ao elemento `<fim>-1`.

`[<inicio>: <fim>]` - permite criar uma sublista com os elementos de `<inicio>` a `<fim>`, onde o elemento na posição `<fim>` não se inclui na nova sublista.

`[<inicio>: ]` - permite criar uma sublista do elemento localizado na posição `<inicio>` até ao final da lista base, onde `<inicio>` se inclui na nova sublista.

`[:-<índice>]` - é semelhante à `[:<fim>]`, porém considerando índices negativos.

```
numeros = [1,2,3,4,5,6,7,8,9,10,11]  
print(numeros[:])  
print(numeros[:-3])  
print(numeros[-1:])  
print(numeros[1:-6])
```

As operações de *slicing* resultariam em sublistas cujos resultados se mostram na Figura 2.

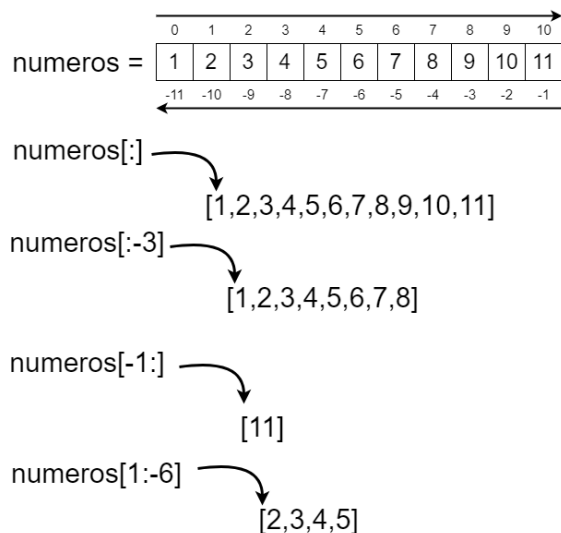


Figura 2. Operações de slicing sobre uma lista

Uma outra estrutura de *slicing* está relacionada a um terceiro campo que serve para indicar o incremento/salto dos índices, possuindo a seguinte estrutura: [*início*]:*fim*:*incremento*].

As operações para essa estrutura, seguem o mesmo padrão que os anteriores porém tendo em conta o último campo que faz referência ao *incremento*.

```
numeros = [1,2,3,4,5,6,7,8,9,10,11]
print(numeros[1::2])
```

O treco anterior, criaria uma sublista com todos os elementos partindo da posição **1** e se estendendo até ao último elemento. Porém, não seriam incluídas todas as posições sequencialmente saltando assim algumas posições num incremento de **2**: [2, 4, 6, 8, 10].

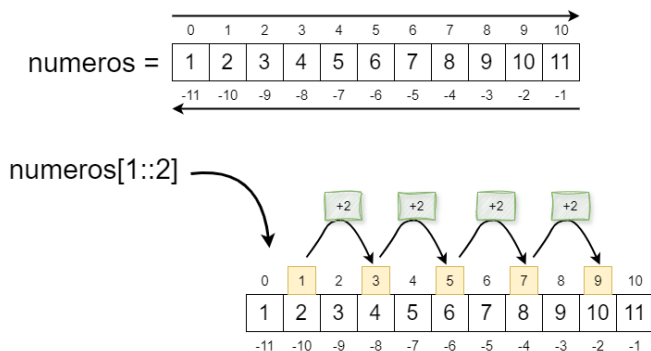


Figura 3. Operações de slicing com pulo/salto

Nota: um pormenor a ter em atenção, está relacionado ao sinal de incremento, pois, ele muda totalmente a direção a percorrer.

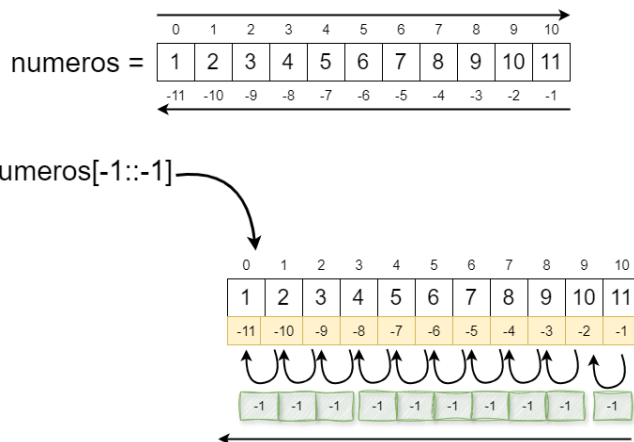


Figura 4. Operações de slicing com salto negativo

2.1.4. Compreensão de lista

Permite a criação rápida de uma lista com base determinadas condições baseadas em um “iterável”. A sua sintaxe é seguinte:

`<novaLista> = [<expressão>(<elemento>) for <elemento> in <iterável> if <condição>]`¹

- `<expressão>`: representa a operação a ser executada (geralmente usando o `<elemento>`) para gerar um novo elemento da sequência desejada.
- `<elemento>`: faz referência ao elemento actual do “iterável” ou sequência (geralmente uma string, tupla ou lista) utilizada como base para gerar a nova lista.
- `<iterável>`: a sequência base a ser percorrida, para então ser gerada a nova lista.
- `<condição>`: parâmetro opcional que ajuda a diversificar as possibilidades para se gerar um novo elemento.

```
numeros = [1,2,3,4,5,6,7,8,9,10,11]
quadrados = [x ** 2 for x in numeros]
print(quadrados)
```

Inicialmente será percorrida a sequência de números (variável `numeros`). A expressão `x**2` permitirá gerar um novo elemento, ou seja, um novo elemento será adicionado à nova lista com base ao resultado dessa expressão. Inicialmente, `x` será 1, depois 2.. até ao último elemento. Isso permitirá criar uma sequência com o quadrado de cada um dos números armazenados na variável `numeros`.

```
listaMatriz = [[j for j in [1,2,3]] for i in [1,2,3]]
print(listaMatriz)
```

Dessa forma, cada novo elemento de `listaMatriz` será uma lista com três elementos.

```
novaLista = [numeros for num in [1,2,3,...,100]
              if num % 5 == 0 if num % 10 == 0]
print(novaLista)
```

Dessa forma, apenas serão considerados (ou adicionados à nova lista) números que cumpram determinada condição. A expressão `[1,2,3,...,100]` não existe em python, apenas foi utilizada para ilustrar uma sequência cujos números começariam de 0 até 100.

¹ Ler a epígrafe 3 (sobre o ciclo **for**) antes de avançar

2.1.5. Atribuição

A atribuição de uma lista usando o operador de atribuição “=” não cria cópia dessa lista, porém passa a referência da lista, permitindo que a nova variável também aponte para essa mesma referência.

```
lista = [1,2,3,4,5]
novaLista = lista
novaLista[2] = 100
print(lista)
print(novaLista)
```

Tecnicamente, alterar a *lista* afetaria a *novaLista* e alterar a *novaLista*, também afetaria a *lista*.

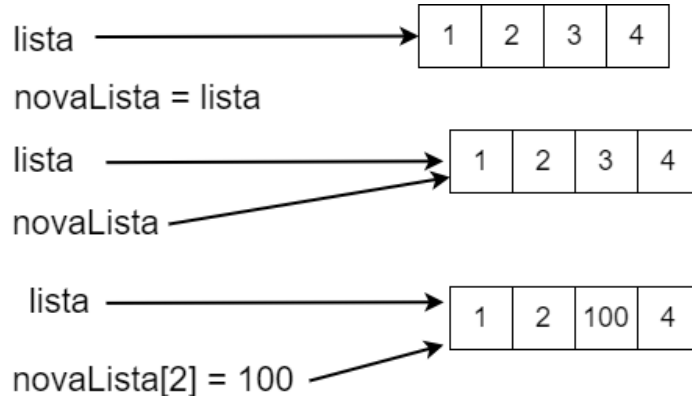


Figura 5. Esquema simples de atribuição com em listas.

2.2. Outras operações sobre as listas

2.2.1. Concatenação e repetição

A concatenação permite juntar/anexar uma lista à outra.

```
lista1 = [1, 2, 3, 0]
lista2 = [4, 5, 6]
resultado = lista1 + lista2
```

Essa operação é bastante intuitiva, permitindo criar uma nova lista com os elementos das listas associadas (duas ou mais listas).

O python permite a criação de uma nova lista a partir de uma lista já existente através do operador “*”.

<sequência>*<n>

O <n> representa o número de repetições a serem feitas para a nova lista, criando assim um nova lista com a concatenação de todas as repetições.

```
elementos = [1,2,3]
print(elementos*3)
```

O resultado desse trecho de código seria impressão de uma nova lista com a seguinte estrutura: [1,2,3,1,2,3,1,2,3].

2.2.2. Aninhamento (nesting)

Consiste em colocar uma coleção (lista, tupla, string, dicionário) dentro de outra. Ou seja, a coleção principal terá como um de seus elementos uma outra coleção.

```
elementos = [[1,2,3,4], 90, "Filipe", (2,4,' ')]
```

Dessa maneira, sempre que se percorrer a lista, quando se chegar numa das colecções armazenadas, deve-se obedecer as regras dessa colecção. Por exemplo, quando se chegar ao elemento na posição 3 (no trecho de código acima), tecnicamente não será possível alterar o valor da tupla (epígrafe 4). Assim sendo, a seguinte operação não seria permitida:

```
elementos = [[1,2,3,4], 90, "Filipe", (2,4, ' ')]
elementos[3][1] = 3
```

O programa tentaria aceder ao elemento na posição 3, que é uma tupla, porém uma tupla não permite alterar internamente o seu valor e por isso produziria um erro de execução.

2.2.2. Funções de uso comum sobre as listas

Sort e Sorted (ordenação)

A primeira delas seria função `.sort()` serve para ordenar uma sequência (números, letras, palavras ou outro tipo de objecto que seja ordenável).

```
numeros = [1,4,2,3,5,8,2]
nomes = ['marta', 'joão', 'zacarias', 'alberto']
numeros.sort()
nomes.sort()
```

No caso dos *numeros*, serão ordenados seguindo a regra de comparação entre inteiros. Para os *nomes* a ordenação será feita com base a ordem alfabética.

Se se desejar uma ordenação na ordem inversa, utiliza-se o parâmetro “reverse = True”, indicando então que a sequência será ordenada na ordem inversa:

```
numeros = [1,4,2,3,5,8,2]
numeros.sort(reverse=True)
```

Um facto interessante sobre a função `.sort()` está no facto de a mesma alterar a estrutura ou configuração dos elementos dentro da lista. Porém, existem situações em que é útil manter a estrutura original da lista e apenas obter a sequência com determinada operação executada (no caso em particular, ordenada). Para evitar isso, utiliza-se a função `sorted()`.

```
numeros = [1,4,2,3,5,8,2]
numerosOrdenados = sorted(numeros)
```

Assim, a lista original se mantém intacta, porém tal operação retornará uma nova lista com os elementos da lista original em ordem. Essa operação também aceita o parâmetro “reverse”

```
numeros = [1,4,2,3,5,8,2]
numerosOrdenados = sorted(numeros, reverse=True)
```

Reverse e Reversed

O método `reverse()` altera a lista permitindo inverter a sequência dos objectos.

```
numeros = [1,4,2,3,5,8,2]
numeros.reverse()
```

O resultado seria: [2,8,5,3,2,4,1].

De forma análoga à função `sort` e `sorted`, existe também a função `reversed()` que permite retornar um objecto iterável com os elementos na ordem inversa sem alterar a sequência base.

```
numeros = [1,4,2,3,5,8,2]
numerosR = reversed(numeros)
for i in numerosR:
    print(i)
```


Dessa forma, o python retornaria um objecto que permitira visitar cada elemento da nova sequência invertida.

Index

O método `.index()` permite retornar a posição da primeira ocorrência de determinado valor dentro de uma sequência.

```
numeros = [1,4,2,3,5,8,2]
print(numeros.index(2))
```

O resultado seria: 2. Pois a primeira ocorrência de 2 (dentro da lista de *numeros*) encontra-se na posição 2.

Count

Esse método é semelhante ao método `.index()`, a diferença está no facto de o método `.count()` procurar por todas as ocorrências do de certo valor dentro de uma sequência e retornar o número de vezes.

```
numeros = [1,4,2,3,5,2,2]
print(numeros.count(2))
```

O resultado seria: 3. Esse é o número de vezes que o valor 2 aparece na lista.

3. Ciclo for e a função range()

3.1. Ciclo for

A função `range()` em python tem um propósito diferente, se comparado às demais linguagens estruturadas. É desenhada especificamente para percorrer sequências ou iteráveis. A sua sintaxe é:

for <variável> **in** <iterável>:
 <declarações>

A <variável> entra como uma variável de controlo que dura enquanto o ciclo estiver em execução. De forma mais simples, receberá sequencialmente o valor de cada elemento da colecção (lista, tupla, dicionários, etc) de maneira a atribuir um valor diferente para cada ciclo.

O <iterável> representa a colecção a ser percorrida. Essa sequência pode ser qualquer tipo de colecção, porem as mais utilizadas são as listas, tuplas e dicionários.

```
lista = [1,4,200,4,3]
for elemento in lista:
    print(elemento)
```

O trecho permitirá ler sequencialmente cada valor da lista, e para cada ciclo a variável `elemento` assumirá um novo valor.

Por se tratar de um ciclo, as palavras **break**, **continue** e **pass** ganham o mesmo significado utilizado pelo ciclo **while**.

```
frase = 'Angola é um país "rico"'
for caractere in frase:
    print(elemento)
```

Dessa forma, é possível percorrer cada um dos elementos da *string* e ter acesso isolado a cada um dos caracteres da string.

3.2. função range()

A função `range()` permite criar um conjunto de mecanismos que permite gerar valores dentro de intervalos numéricos, e é comumente utilizada como base para percorrer um ciclo (mais comumente o ciclo **for**).

`range(<inicio>,<fim>)` – permite gerar uma sequência numérica de `<inicio>` até `<fim> - 1`.

```
for numero in range(1,100):  
    print(numero)
```

O trecho acima permite gerar uma sequência de números de 1 a 99. Um facto interessante, acontece quando se omite um dos parâmetros, por exemplo `range(10)` vai gerar uma sequência de números tendo o único argumento de `range` como fim da sequência, ou seja, de 0 a 9.

Uma outra abordagem à função `range()` seria análoga aos métodos de subdivisão de listas (epígrafe 2.1.3 sobre slicing). Dessa forma, é possível gerar intervalos numéricos com três parâmetros. O último parâmetro também faz referência ao salto/pulo:

`range(<inicio>,<fim>,<salto>)`

```
for numero in range(1,100,3):  
    print(numero)
```

O trecho de código anterior permitiria mostrar todos os números múltiplos de 3, por causa da condição de salto/pulo.

Nota: diferente das técnicas de *slicing*, se o parâmetro de salto for negativo, ela vai gerar uma sequência de números na direção oposta.

4. Tuplas

As listas são úteis para armazenar coleções de dados que podem alterar ao longo do tempo de execução do programa. Porém, algumas vezes é útil ter um conjunto de itens que serão imutáveis durante o tempo de execução do programa. As *tuplas*, são úteis para situações dessa natureza.

As tuplas, são coleções imutáveis de objectos, separados por “vírgulas” e delimitados por parênteses “(” e “)” no início e fim da sequência.

```
tupla = (1, 2, 3, 4)  
print(tupla[2])
```

O acesso às tuplas é semelhante ao acesso às listas. Mas precisamente para o exemplo acima, faz-se o acesso por indexação aos valores da tupla.

➔ Quase todas as operações possíveis numa lista (epígrafe 2), também serão possíveis numa tupla. Excepto aquelas que alteram os valores das tuplas internamente, pois uma tupla é imutável. Tais operações seriam: concatenação, aninhamento (nesting), repetição, slicing, obter o comprimento, conversão de para lista e percurso dentro de um ciclo.

```
minhaTupla = (1,2,3,4)  
minhaTupla = (3, 5)
```

Tecnicamente não se altera a tupla, apenas se muda o valor da variável *minhaTupla* para uma nova tupla, diferente da anterior.

O python utiliza o operador **del** sobre uma tupla, apenas para eliminar a tupla como um todo. Essa operação nunca deve ser usada para eliminar um elemento numa posição em particular.

4.1. Conversão de lista para tupla e vice-versa

O processo é bastante simples e intuitivo. Quando se tem tupla, utiliza-se o método `list(<tupla>)`, quando se tem lista utiliza-se o método `tuple(<lista>)`.

```
minhaTupla = (1,2,3,4)
minhaLista = list(minhaTupla)
segundaTupla = tuple(minhaLista)
```

Bibliografia

- [1] M. Lutz, Learning Python, Fifth Edition, Califórnia: O'Reilly Media, Inc., 2013.
- [2] F. Python, Luciano Ramalho, Califórnia: O'Reilly Media, Inc., 2014.
- [3] E. Matthes, Python Crash Course, 3RD EDITION, Califórnia : No Starch Press, Inc., 2023.
- [4] G. f. Geeks, "Python Lists," 2024. [Online]. Available: <https://www.geeksforgeeks.org/python-lists/>.